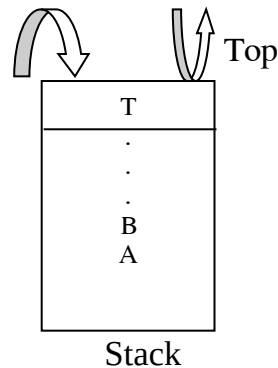


Bài 3 -Tiếp chương 2 .CÁC CẤU TRÚC DỮ LIỆU CƠ BẢN

2.2.NGĂN XẾP - STACK

2.2.1. Khái niệm

Ngăn xếp là một cấu trúc dữ liệu trừu tượng, xử lý theo kiểu vào sau ra trước (last in first out — LIFO) nó là một danh sách hay một dãy bản ghi- cấu trúc, trong đó mỗi phép toán thêm vào hay bớt đi đều thực hiện ở một đầu gọi là đỉnh ngăn xếp.



Một số ứng dụng kiểu ngăn xếp :

- chuyển đổi cơ số
- gọi và kết thúc các thủ tục
- Lịch sử duyệt web
- Dãy Undo trong bộ soạn thảo văn bản
- Kiểm tra tính hợp lệ của dấu ngoặc đơn trong biểu thức toán học
- Ứng dụng trong cài đặt chương trình Compiler
- Tính giá trị biểu thức toán học
- Quay lui (Backtracking)
- Khử đệ quy
-

Xét bài toán đổi một số ở hệ đếm cơ số 10 sang hệ 2 theo cách sử dụng ngăn xếp .

1.Nhập số cần dịch đổi , tạo 1 ngăn xếp rỗng.

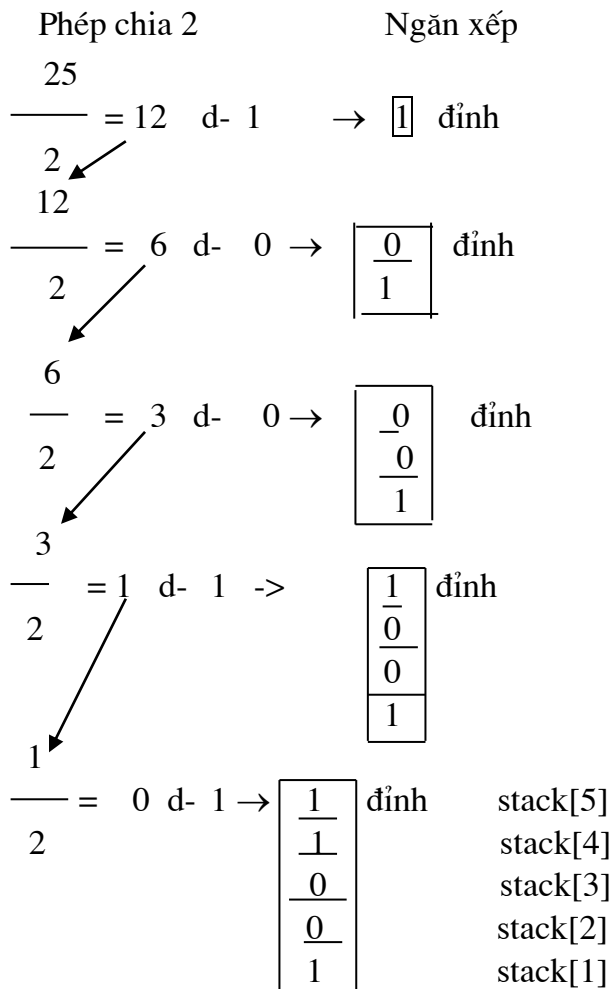
2. Khi mà số $\neq 0$ thì làm các việc :

- a)Tính $Số_{d-} = Số \% 2$;
- b)Nhớ lấy số $d-$ (gửi số $d-$ vào đỉnh ngăn xếp);
- c)Thay số bằng giá trị phần nguyên của phép chia nguyên của số cho 2
($So = so/2$)

3.Khi thương số = 0 thì hiện kết quả : lần lượt hiện các ô nhớ chứa số $d-$, đi ngược từ cuối đến đầu, sẽ được biểu diễn hệ 2 của số thập phân ban đầu. Để làm việc này, trước hết kiểm tra, nếu ngăn xếp không rỗng thì :

- a)Lấy số $d-$ từ đỉnh ngăn xếp
- b)Hiện lên màn hình số $d-$ vừa lấy

Ta có thể biểu diễn theo sơ đồ sau :



Để khỏi phải dịch chuyển xuống/lên các phần tử mỗi khi thêm/bớt phần tử ở đỉnh, ta tăng/giảm chỉ số của các phần tử.

```
for(int i =5;i>=1;i--) cout<<stack[i];
```

Khi hiện kết quả ra màn hình, sẽ hiện lần lượt giá trị phần tử ở đỉnh.

Kết quả được biểu diễn dạng nhị phân của số 25 là 11001.

Viết theo công thức

$$25_{10} = 2^4 * 1 + 2^3 * 1 + 2^2 * 0 + 2^1 * 0 + 2^0 * 1$$

2.2.2. Các thao tác cơ bản đối với ngăn xếp

Cách đơn giản nhất để thể hiện stack là sử dụng mảng 1 chiều, chẳng hạn ký hiệu stack[1..n]. Phần tử đầu tiên (hay đỉnh ngăn xếp) là stack[i] tại thời điểm thứ i, phần tử thứ 2 là stack[i-1], ...phần tử dưới cùng là stack[1]. Ta dùng 1 biến **top** để chỉ phần tử ở đỉnh ngăn xếp.

Khi làm việc với ngăn xếp, các thao tác cơ bản là :

IsFull(**sizestack**): Kiểm tra ngăn xếp đã đầy chưa;

IsEmpty() — Kiểm tra ngăn xếp trống hay không;

Push(int stack[], int gia_tri, int **sizestack**) – Thêm phần tử vào đỉnh ngăn xếp;

Pop() - Bớt đi phần tử ở đỉnh ngăn xếp, số phần tử giảm đi 1;

Top(Stack)- Lấy ra giá trị của phần tử đỉnh ngăn xếp, số phần tử không thay đổi;

Size(): Tính số lượng phần tử stack đang có.

2.2.3. Cài đặt ngăn xếp bằng mảng

Dưới đây sẽ giới thiệu một số hàm cho một số thao tác đối với mảng ngăn xếp.

Sử dụng mảng 1 chiều **stack** kiểu int làm ngăn xếp, một biến **top** lưu chỉ số của phần tử ở đỉnh ngăn xếp, một biến **sizestack** lưu kích thước của ngăn xếp.

2.2.3.1. Kiểm tra stack có đầy không (IsFull)

Hàm này sẽ kiểm tra xem **stack** hiện tại đã đầy hay chưa. Nếu chỉ số **top** của stack đang bằng với **sizestack-1** là ngăn xếp đã đầy.

```
bool IsFull(int sizestack)
{
    if(top >= sizestack - 1){ return true; }
    else
    { return false;}
}
```

2.2.3.2. Kiểm tra stack rỗng(IsEmpty)

Nếu stack đang không có phần tử nào, ta sẽ gán chỉ số **top = -1** để đánh dấu. Như vậy, để kiểm tra stack có đang rỗng hay không ta chỉ cần so sánh giá trị **top** có phải -1 hay không mà thôi.

```
bool IsEmpty()
{
    if(top == -1){ return true; }
    else
    { return false; }
}
```

2.2.3.3. Thêm phần tử vào đỉnh stack(Push)

Chúng ta chỉ có thể thêm phần tử vào đỉnh ngăn xếp khi nó chưa đầy. Nếu ngăn xếp đã đầy, sẽ đưa ra thông báo và không thêm nữa. Ngược lại, sẽ tăng **top** lên một đơn vị và gán giá trị cho phần tử tại chỉ số **top**.

```
void Push(int stack[], int gia_tri, int sizestack )
{
    if(IsFull(sizestack) == true)
        {cout<< "\nStack is full – Ngan xep da day ! \n"; }
    else
    { ++top; stack[top] = gia_tri; }
}
```

2.2.3.4. Xóa phần tử khỏi đỉnh stack(Pop)

Ta chỉ có thể xóa phần tử (pop) khỏi đỉnh ngăn xếp khi nó không trống. Xóa phần tử ở đỉnh thực chất là không quan tâm đến phần tử đỉnh hiện tại nữa, chỉ số **top** giảm đi 1 chứ không có phép xóa-delete nào xảy ra cả. Nếu ngăn xếp trống, sẽ đưa ra thông báo và không thực hiện xóa. Ngược lại, sẽ giảm giá trị **top** đi một đơn vị.

```
void Pop(){
```

```

if(IsEmpty() == true)
    {cout<< " \n Ngan xep trong ! \n";}
else
    { -- top; }
}

```

2.2.3.5. Lấy giá trị phần tử ở đỉnh stack(top)

Để lấy giá trị phần tử ở đỉnh ngăn xếp, ta có thao tác:

```

int Top(int stack[])
{
    return stack[top];
}

```

2.2.3.6. Tính số lượng phần tử stack đang có(Size)

Biến **top** lưu chỉ số lớn nhất của ngăn xếp, việc tính số lượng phần tử stack đang có của stack là dùng hàm size() của C++ :

```

int Size()
{
    return top + 1;
}

```

Kết nối lại, ta sẽ có chương trình đầy đủ :

```

#include <iostream>
using namespace std;

int top = -1; //Biến toàn cục Global
//Khai báo các nguyên mẫu hàm
bool IsFull(int sizestack);
bool IsEmpty();
void Push(int stack[], int gia_tri, int sizestack);
void Pop();
int Top(int stack[]);
int Size();
int main(){
    int sizestack ;
    cout<<" \n  Nhập lại kích thước stack :\n"; cin>>sizestack;
    cout<<"  Kích thước sizestack mới là : "<<sizestack<<"\n";
    system("pause");
    int stack[sizestack]; .
    Push(stack, 10, sizestack); // Thêm số 10 vào đỉnh ngăn xếp
}

```

```

cout<<"\n\n";
cout<<"    Kích thước hiện tại của Ngăn xếp : "<<Size()<<"\n";
Push(stack, 15, sizestack); //Thêm số 15 vào đỉnh ngăn xếp
Push(stack, 30, sizestack); //Thêm số 30 vào đỉnh ngăn xếp
cout<<"    Kích thước hiện tại của Ngăn xếp : "<<Size()<<"\n";
Push(stack, 45, sizestack); //Thêm số 45 vào đỉnh ngăn xếp
cout<<"    Giá trị vừa nhập sau cùng : "<<stack[top]<<"\n";
cout<<"    Hiện giá trị ở đỉnh Ngăn xếp : "<<Top(stack)<<"\n";
cout<<"    Hiện các giá trị Ngăn xếp : \n" ;
int i;
for(i=top;i>=0;i--) { cout<<"    " << stack[i]<<"\n"; }
//Xóa tất cả các phần tử của Ngăn xếp
for(int i = 0 ; i < 3;i++) Pop();
cout<<"    Kích thước hiện tại của Ngăn xếp : "<<Size()<<"\n";
Pop();
system("pause");
return 0;
}

// Các định nghĩa hàm
bool IsFull(int sizestack)
{ if(top >= sizestack - 1){ return true; }
  else
{ return false; }
}

bool IsEmpty()
{ if(top == -1){ return true; }
  else
{ return false; }
}

void Push(int stack[], int gia_tri, int sizestack)
{ if(IsFull(sizestack) == true)
    { cout<<" Stack is full. Overflow condition! \n";}
  else
    { ++ top; stack[top] = gia_tri; }
}

```

```

void Pop()
{ if(IsEmpty() == true)
    { cout<<" Stack is empty. Underflow condition!"; }
  else
    { -- top; }
}

```

```

int Top(int stack[])
{ return stack[top]; }

```

```

int Size()
{ return top + 1; }

```

//*****

Kết quả chạy chương trình trên với sizestack =3

```

Kich thuoc hien tai cua Ngan xep :1
Kich thuoc hien tai cua Ngan xep :3
Stack is full. Overflow condition!
Gia tri cuoi :30
Hien gia tri o dinh Ngan xep :30
Hien cac gia tri Ngan xep :
30
15
10
Kich thuoc hien tai cua Ngan xep :0
Stack is empty. Underflow condition!

```

Kết quả chạy chương trình với sizestack =4 :

```

Nhap lai kich thuoc stack :
4

Kich thuoc hien tai cua Ngan xep :1
Kich thuoc hien tai cua Ngan xep :3
Gia tri cuoi :45
Hien gia tri o dinh Ngan xep :45
Hien cac gia tri Ngan xep :
45
30
15
10
Kich thuoc hien tai cua Ngan xep :1
Press any key to continue . . .

```

Sử dụng các hàm vừa xét ở trên, viết chương trình đổi một số nguyên bất kỳ từ hệ 10 sang hệ nhị phân. Ta sử dụng mảng cấu trúc để biểu diễn ngăn xếp và viết chương trình với một số khai báo như sau :

```
#include <iostream>
#include <iomanip>
using namespace std;

#define limit 50

struct stack // Định nghĩa cấu trúc stack
{
    int top;
    int nx[limit];
};

void Create(stack &s)// Khởi tạo ngăn xếp
{
    s.top=-1;
}

bool IsFull(stack s)
{
    if( s.top == limit - 1){ return true; }
    else
        { return false; }
}

bool IsEmpty(stack s)
{
    if( s.top == -1){ return true; }
    else
        { return false; }
}

void Push(stack &s, int gia_tri)
{
    if(IsFull(s))
        cout<<" Ngăn xếp đầy - Stack is full. Overflow condition! \n";
    else
        { s.top++; s.nx[s.top] = gia_tri; }
}

int Pop(stack &s)
{
    if(IsEmpty(s))
        { cout<<" Ngăn xếp rỗng - Stack is empty !"; }
    else
        return s.nx[s.top--];
}

int main()
{
```

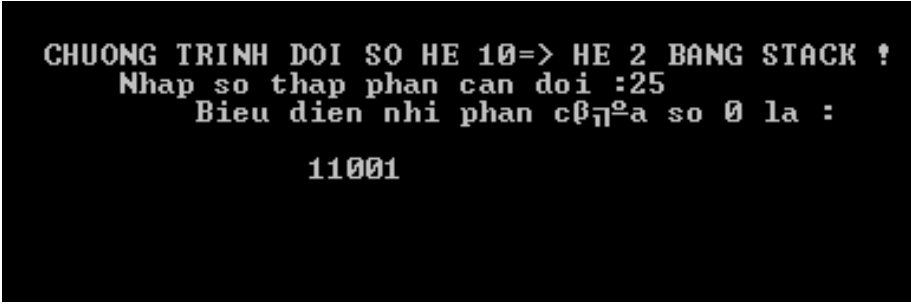
```

cout << "DOI SO HE 10=> HE 2 BANG STACK !" << endl;
stack s;
Create(s);
int a,a1,p;
do
    { cout<<"      Nhap so thap phan can doi :";
      cin>>a;
    }while (a<=0);
a1=a;
while (a!=0){
    p=a%2;
    Push(s,p);
    a=a/2;
}

cout<< "      Bieu dien nhi phan cua so "<<a1<< " la : \n\n";
cout<<setw(20);
while(IsEmpty(s)== 0) cout<<Pop(s); cout<<"\n";
system("pause");
return 0;
}

```

Kết quả chạy chương trình :



```

CHUONG TRINH DOI SO HE 10=> HE 2 BANG STACK !
  Nhap so thap phan can doi :25
  Bieu dien nhi phan cua so 25 la :
      11001

```

2.2.4. Thư viện chuẩn chứa các hàm thao tác trên ngăn xếp

Để sử dụng các thao tác cơ bản trên ngăn xếp như- những hàm mẫu có sẵn trong thư viện hàm mẫu STL của C++, ta phải khai báo thư viện stack ở phần đầu chương trình :

```
#include <stack>
```

Sau đó khai báo biến: stack<kiểu dữ liệu> <tên biến>;

Ví dụ : stack<int> s;

Các thủ tục trong thư viện chuẩn stack :

1) Đưa(đẩy) phần tử x vào stack : s.push(x);

2) Lấy phần tử đỉnh của stack : int u = s.top();

Thủ tục này chỉ lấy phần tử đỉnh ra thôi chứ không xóa.

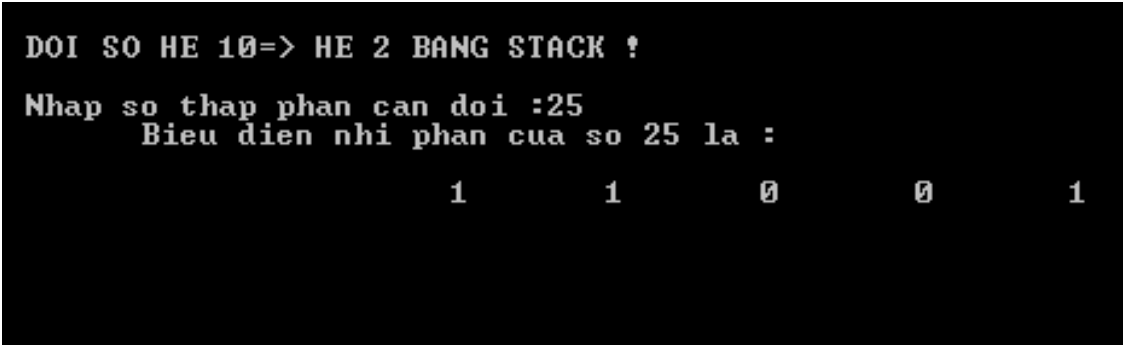
3) Xóa phần tử đỉnh: s.pop(); Thủ tục này chỉ xóa phần tử đỉnh, không trả về giá trị.

4) Kiểm tra rỗng: trả về true hoặc false : s.empty();

Chương trình có sử dụng thư viện chuẩn stack, không cần phải viết các hàm push(),top(),pop(),empty()

```
#include <iostream>
#include <stack> //sử dụng thư viện stack
#include <iomanip> //điều khiển việc hiện dữ liệu lên màn hình
using namespace std;
int main()
{ cout << "      \n\n    DOI SO HE 10=> HE 2 BANG STACK !" << endl;
  stack<int> s;
  int a,a1,p;
  do
  { cout<<"    \n    Nhập số thập phân cần đổi :"; cin>>a;
    }while (a<=0);
  a1=a;
  while (a!=0){
    p = a%2;
    s.push(p);
    a = a/2;
  }
  cout<< "    \n    Biểu diễn nhị phân của số "<<a1<< " là : \n\n";
  cout<<setw(20);
  while(!s.empty()){
    cout<<"\t"<<s.top(); s.pop();    }
  cout<<"\n";
  system("pause");
  return 0;
}
```

Kết quả chạy chương trình :



```
DOI SO HE 10=> HE 2 BANG STACK !
Nhập số thập phân cần đổi :25
    Biểu diễn nhị phân của số 25 là :
                1        1        0        0        1
```

2.2.5..Lớp ngăn xếp – stack

Trong định nghĩa lớp sẽ có các thành viên thuộc tính và phương thức. Khi xây dựng stack, có các hàm cơ bản là push đẩy phần tử vào đỉnh ngăn xếp và pop lấy phần tử ở đỉnh ngăn xếp, 2 hàm phương thức này có thể viết bên trong hay bên ngoài định nghĩa lớp. Để khởi tạo một đối tượng mới cho lớp Stack cần dùng thêm Hàm tạo Constructor.

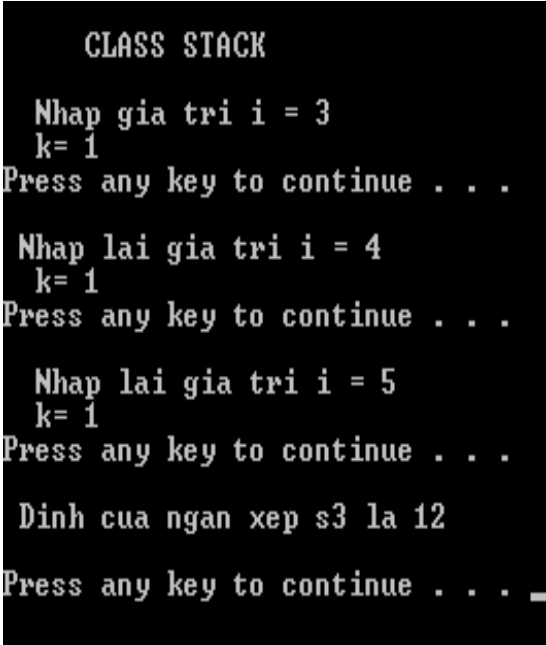
```
#include <iostream>
using namespace std;

class Stack
{ private:
    int s[20];
    int k;
public:
    Stack()// constructor
    {    k = 0;  }
    void push(int x);
    int pop()
    {    return s[- k];  }
};// hết định nghĩa class Stack

void Stack::push(int x)
{    s[k++] = x;
    cout<<" k= "<<k<<" \n";
    system("pause");
}

int main()
{    cout << "\n\n CLASS STACK \n" << endl;
    int i;
    Stack s1,s2,s3;
    cout << " Nhập gia tri i = " ;cin>>i;
    s1.push(i);
    cout << "\n Nhập lai gia tri i = " ;cin>>i;
    s2.push(s1.pop() + i);
    cout << "\n Nhập lai gia tri i = " ;cin>>i;
    s3.push(s2.pop() + i);
    cout << "\n Dinh cua ngan xep s3 la
"<<s3.pop()<< endl;
    cout << "\n" ;
    system("pause");
    return 0;
}
```

Kết quả chạy chương trình :



```
CLASS STACK
Nhập gia tri i = 3
k= 1
Press any key to continue . . .
Nhập lai gia tri i = 4
k= 1
Press any key to continue . . .
Nhập lai gia tri i = 5
k= 1
Press any key to continue . . .
Dinh cua ngan xep s3 la 12
Press any key to continue . . .
```

2.2.6. Ngăn xếp liên kết

Ngăn xếp liên kết là một kiểu riêng của danh sách liên kết đã xét ở phần danh sách – LIST.

Khi chỉ có nút đầu tiên trong danh sách liên kết đ-ọc truy nhập trực tiếp thì ta có một cơ chế làm việc nh- là ngăn xếp.

Với những suy xét trên, chúng ta sẽ sử dụng danh sách liên kết để cài đặt ngăn xếp liên kết, mỗi thành phần của ngăn xếp liên kết được coi là một nút của danh sách liên kết. Để làm việc với ngăn xếp liên kết cần nắm vững kiến thức về cấu trúc – struct, về con trỏ – pointer và về danh sách liên kết – linked list.

Những nội dung cần thực hiện :

- Khai báo cấu trúc struct nút của ngăn xếp liên kết
- Khai báo cấu trúc ngăn xếp liên kết
- Khởi tạo nút mới ở đỉnh ngăn xếp liên kết
- Khai báo hàm đẩy dữ liệu vào đỉnh ngăn xếp liên kết – ThemvaoDinh
- Khai báo hàm kiểm tra ngăn xếp liên kết có rỗng không - isEmpty
- Khai báo hàm Xóa nút đỉnh ngăn xếp liên kết – XoanutDinh
- Khai báo hàm lấy dữ liệu ở đỉnh ngăn xếp liên kết – Top
- Sử dụng các hàm cho một ứng dụng cụ thể

2.2.6.1. Khai báo cấu trúc Node

Ký hiệu Node cho tên kiểu cấu trúc nút của ngăn xếp liên kết :

```
struct Node
{   int  du_lieu;
    Node* next; //con trỏ liên kết có kiểu Node
};
```

2.2.6.2. Khai báo cấu trúc ngăn xếp liên kết và khởi tạo ngăn xếp

Ngăn xếp liên kết khác với danh sách liên kết ở chỗ mọi thao tác xử lý đều ở đỉnh ngăn xếp, khi tạo một ngăn xếp mới, chỉ cần một con trỏ chỉ vào đỉnh ngăn xếp, các phần tử Node nối tiếp từ đỉnh sẽ có phần dữ liệu và liên kết next cho phần tử sau. Vì vậy cấu trúc **LinkedStack** lưu trữ địa chỉ phần tử đầu -đỉnh (head) có dạng :

```
struct LinkedStack
{ Node* head;
};
```

Tiếp đến **khởi tạo ngăn xếp liên kết rỗng ban đầu**

```
void KhoiTaoStack(LinkedStack& s)
{ s.head=NULL; //Lúc đầu Ngăn xếp rỗng nên được gán NULL
}
```

2.2.6.3. Khởi tạo nút mới để nhận dữ liệu cho ngăn xếp liên kết

```
Node* KhoitaoNode(int du_lieu0)
{
    Node* p = new Node; //tạo nút p mới và cấp phát bộ nhớ động cho p
    p->du_lieu = du_lieu0; //khởi gán giá trị dữ liệu ban đầu du_lieu cho p
    p->next = NULL; //nút mới tạo chưa trở vào đâu cả nên next=NULL
    return p; // trả về địa chỉ của nút mới được cấp phát
}
```

2.2.6.4. Khai báo hàm isEmpty - kiểm tra ngăn xếp liên kết có rỗng không

```
bool isEmpty(LinkedStack s)
{
    return (s.head == NULL);
}
```

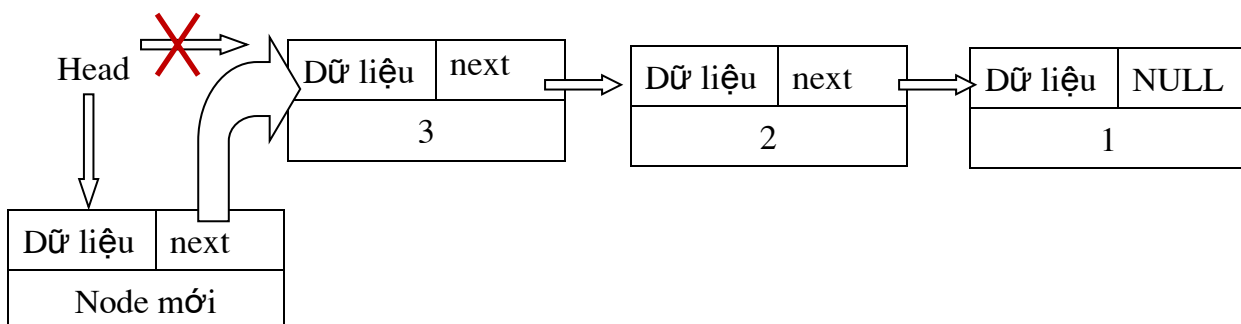
2.2.6.5. Thêm phần tử vào đỉnh ngăn xếp liên kết

- Kiểm tra, nếu ngăn xếp liên kết rỗng thì phần tử mới hiện thời là đầu ngăn xếp liên kết, ở đỉnh ngăn xếp;

- Nếu ngăn xếp không rỗng thì phải đẩy nút đang ở đỉnh xuống rồi đưa nút mới hiện thời vào đỉnh ngăn xếp liên kết.

Việc thêm phần tử ở đỉnh ngăn xếp liên kết tương tự như thêm phần tử vào đầu danh sách liên kết đơn đã xét ở chương 2.

Sơ đồ thêm phần tử vào đầu danh sách liên kết – tương tự như thêm phần tử vào đỉnh ngăn xếp liên kết, ngăn xếp liên kết được trình bày hàng ngang thay vì cột dọc:

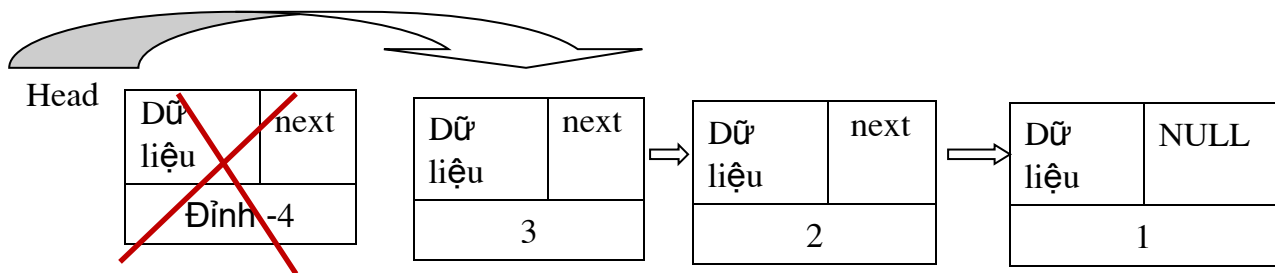


Ký hiệu **pht* là con trỏ trỏ đến nút mới hiện thời, ta có hàm sau :

```
void ThemvaoDinh(Linkedstack &s, int du_lieu0)
{
    Node* pht = KhoitaoNode(int du_lieu0); // Tạo nút mới hiện thời
    if (isEmpty(s)) // kiểm tra stack rỗng không
    { s.head = pht; //nếu ngăn xếp rỗng, nút mới hiện thời sẽ là đỉnh ngăn xếp l.kết }
    else //nếu ngăn xếp liên kết không rỗng, liên kết của nút mới hiện thời sẽ trở
        // đến đỉnh ngăn xếp liên kết, đỉnh ngăn xếp sẽ nhận nút hiện thời :
    { pht->next = s.head;
      s.head = pht;
    }
}
```

2.2.6.6.Xóa phần tử ở đỉnh ngăn xếp liên kết

Xóa phần tử ở đỉnh ngăn xếp liên kết tương tự như xóa nút đầu của danh sách liên kết đơn:



Bước 1. Kiểm tra xem ngăn xếp liên kết có rỗng hay không, nếu rỗng, trả về kết quả là 0 và thoát ra, nếu không rỗng thì tiếp bước 2;

Bước 2. Lưu lại nút đỉnh, gán head bằng next của nút đỉnh rồi xóa nút đỉnh;

```
int XoaDinh(LinkedStack &s)
{
    if (!isEmpty(s))
    {
        Node *x = s.head; //khai báo tạo Node x và Lưu lại nút đỉnh
        int a = x->du_lieu; //khai báo tạo một biến a nhận dữ liệu nút đỉnh
        s.head = x->next; // đỉnh mới của ngăn xếp trở tới nút mà đỉnh trước trở đến
        delete(x); // thực hiện xóa nút x
        cout<<"Xóa thành công !!";
        return a; // trả về dữ liệu nút đỉnh đã xóa
    }
    else
    {
        cout << "Stack rỗng!" << endl;
        return 0; }
}
```

2.2.6.7.Hiện dữ liệu của phần tử đỉnh ngăn xếp liên kết

Bước 1. Kiểm tra ngăn xếp liên kết, nếu rỗng thì thoát ra, nếu không thì xuống bước 2;

Bước 2.Hiện dữ liệu của đỉnh

```
int Dinh(LinkedStack s)
{
    if (!isEmpty(s)) // kiểm tra stack , nếu không rỗng thì hiện dữ liệu
    {
        return s.head->du_lieu; }
    else //nếu rỗng :
    {
        cout << "Stack rỗng!" << endl; return 0; }
}
```

2.2.6.8.Hiện cả ngăn xếp liên kết

Mọi thao tác với dữ liệu ngăn xếp liên kết đều thực hiện ở đỉnh ngăn xếp, kể cả việc hiện dữ liệu cả ngăn xếp liên kết.

```
{ if (s.head != NULL)
{ Node* node = s.head; //Khai báo nút mới và được gán nút định
while (node != NULL)
{cout<< node->du_lieu<<" ";
node = node->next;
}
}
```

```
cout << " \n\n\t\tCAC CHUC NANG CUA CHUONG TRINH ";
cout << " \n1. Nhap du lieu vao ngan xep lien ket ";
cout << " \n2. Hien Dinh ngan xep lien ket ";
cout << " \n3. Xoa phan tu Dinh ngan xep lien ket ";
cout << " \n4. Hien ngan xep lien ket ";
cout << " \n0. Thoat Ra ";
cout<<"\n\t\tNhap lua chon (1/2/3/4/0) : ";
cin>>chon;
```

```

{
    .....
    switch(chon)
    {case 0: break;
      case 1:
          ....
          break;
      case 2:
          .....
          break;
      .....
    }
}
while(chon);

```